Power BI

Interview Questions & Answers

Complete Interview Preparation Guide • 120+ Questions • Categorized

Prepared by: Vaibhav Rajasha

Data Analyst

GitHub: vaibhavdhakulkar | LinkedIn: vaibhavdhakulkar25

SECTION 1: Power BI Fundamentals

Q1. What is Power BI?

Answer:

Power BI is a Business Intelligence (BI) and Data Visualization tool developed by Microsoft. It connects to many data sources, transforms raw data, and turns it into interactive dashboards and reports. Both technical and non-technical users can use it to make data-driven decisions.

Q2. Why should we use Power BI?

Answer:

Power BI makes it easy to connect to various data sources, clean and transform data, and create beautiful visuals — all without writing code. It helps businesses make faster and smarter decisions by turning raw data into actionable insights.

Q3. What are the major components of Power BI?

Answer:

Power BI has several key components:

- Power BI Desktop — Free Windows app to build reports and data models.
- Power BI Service — Cloud platform (app.powerbi.com) to publish, share, and collaborate.
- Power BI Mobile — iOS/Android app to view dashboards on the go.
- Power BI Gateway — Bridge for refreshing on-premises data from the cloud.
- Power BI Report Server — On-premise version for organizations that cannot use the cloud.
- Power BI Embedded — For developers to embed reports in custom applications.
- Power BI Report Builder — For creating pixel-perfect paginated reports.

Q4. What is the difference between Power BI Desktop and Power BI Service?

Answer:

Power BI Desktop is a local Windows application used to design, build, and model reports. Power BI Service is the cloud-based platform where reports are published for sharing, collaboration, and scheduled refresh. Think of Desktop as the 'building tool' and Service as the 'sharing platform'.

Q5. What are the three core engines inside Power BI Desktop?

Answer:

Power BI Desktop is powered by three engines:

- Power Query — The ETL (Extract, Transform, Load) engine for data cleaning and shaping.
- Power Pivot — The data modeling engine. Manages relationships and DAX calculations using VertiPaq in-memory database.
- Power View — The visualization engine. The report canvas where charts, tables, and dashboards are built.

Q6. What is the standard workflow in Power BI?

Answer:

The standard end-to-end workflow in Power BI is:

- Connect — Import data from Excel, SQL, Web, APIs, etc.
- Transform — Clean and reshape data in Power Query.
- Model — Define relationships between tables in Model View.
- Calculate — Write DAX measures and calculated columns.
- Visualize — Build charts, slicers, and dashboards in Report View.
- Publish — Upload to Power BI Service and share with stakeholders.
- Refresh — Schedule automatic data refresh to keep reports current.

Q7. What are the available views in Power BI Desktop?

Answer:

There are three main views in Power BI Desktop:

- Report View — Design interactive visuals and report pages.
- Table (Data) View — Inspect data in row-column format, sort, and apply conditional formatting.
- Model View — Create and manage relationships between tables.

Q8. What is Self-Service BI (SSBI) and how does Power BI support it?

Answer:

Self-Service BI allows business users (non-IT, non-programmers) to independently connect to data, create reports, and generate insights without depending on an IT team. Power BI enables this through its drag-and-drop interface, guided Power Query transformations, and point-and-click DAX measures.

Q9. What is the difference between a Report and a Dashboard in Power BI?

Answer:

Reports and dashboards serve different purposes in Power BI:

- Report: Can have multiple pages, uses one dataset, created in Desktop & Service, best for detailed analysis.
- Dashboard: Only a single page, can pull from multiple reports/datasets, created only in Service, best for high-level KPI overview.
- Use a Report when you need detailed analysis with filters and drill-down.
- Use a Dashboard when you need a quick overview of key metrics for executives.

Q10. What are the different Power BI license versions?

Answer:

Power BI has the following license tiers:

- Power BI Desktop — Free. For building reports locally, cannot share.
- Power BI Pro (~$10/user/month) — Full sharing and collaboration in Service.
- Power BI Premium (~$20/user/month or $4,995/capacity/month) — Enterprise scale, dedicated storage up to 100TB, allows free users to view shared content.
- Power BI Embedded — For developers embedding reports in custom applications.

Q11. What is a content pack in Power BI?

Answer:

A content pack is a pre-built, ready-to-use collection of dashboards, reports, and data connections created by Microsoft or third-party providers (e.g., Google Analytics, Salesforce). You use them when you need to quickly connect to a popular service and get instant insights without building reports from scratch. Content packs are customizable after import.

Q12. What is the difference between Power BI and Tableau?

Answer:

Key differences between the two tools:

- Language: Power BI uses DAX for measures; Tableau uses MDX.
- Data Size: Power BI handles small-to-medium datasets better; Tableau handles very large datasets.
- User Level: Power BI is suitable for both beginners and experts; Tableau is more expert-oriented.
- UI: Power BI has a simpler, familiar Office-like interface; Tableau has a more complex UI.
- Cost: Power BI is generally more affordable for organizations already on Microsoft stack.

SECTION 2: Power Query — Data Transformation (ETL)

Q13. What is Power Query and what is its role in Power BI?

Answer:

Power Query is the ETL (Extract, Transform, Load) engine in Power BI. It allows you to connect to any data source, clean messy data, and reshape it into a structured format before loading it into the data model. It uses M Language for transformations. Every step is recorded and replayed automatically on data refresh.

Q14. What is the difference between Merge and Append in Power Query?

Answer:

These are two different ways to combine data:

- Merge — Combines two tables side-by-side based on matching key columns. This is equivalent to a SQL JOIN.
- Append — Stacks two tables vertically by adding rows from one table below another. This is equivalent to SQL UNION.
- Example: Merge customer table with sales table on Customer_ID (JOIN). Append January sales with February sales (UNION).

> **Tip:** Use Merge when you want to add more columns. Use Append when you want to add more rows.

Q15. What are Applied Steps in Power Query?

Answer:

Applied Steps is an audit log of every transformation performed in Power Query. Each action (filter, rename, type change) is saved as a step that can be reviewed, edited, reordered, or deleted. When data is refreshed, Power BI re-runs all steps automatically — making the process reproducible and automated. You can see them in the Query Settings pane on the right side of the Power Query Editor.

Q16. What is Query Folding in Power BI?

Answer:

Query Folding is the process where Power Query translates M Language transformation steps into native SQL queries that are executed directly on the database server — rather than loading all data into Power BI first. This significantly improves performance for large datasets. Query folding works for operations like filtering rows, removing columns, grouping, and merging. The translation from M Query to SQL and vice versa is what is called query folding.

Q17. What is the difference between Pivot and Unpivot in Power Query?

Answer:

Pivot and Unpivot change the shape of your data:

- Pivot — Converts unique row values into separate columns (wide format). Example: Converting months that are in rows into separate Month columns.
- Unpivot — Converts multiple columns into key-value row pairs (tall/normalized format). Example: Converting column headers like Jan, Feb, Mar into a single 'Month' column with values.
- Unpivot is essential for making wide tables compatible with Power BI's visualization engine, which requires normalized data.

Q18. What is the difference between Power Query and DAX?

Answer:

Both are used in Power BI but at different stages and for different purposes:

- Power Query (M Language): Used for data preparation and transformation (ETL). Runs during data load and refresh. Best for cleaning, merging, and reshaping data.
- DAX (Data Analysis Expressions): Used for calculations and aggregations. Runs during report rendering. Best for measures, time intelligence, and KPI calculations.
- Rule of Thumb: Push as much work as possible to Power Query. DAX is for analysis logic, not ETL work.

Q19. What are common tasks performed in the Power Query Editor?

Answer:

Common transformation tasks in the Power Query Editor include:

- Connect to data sources (Excel, SQL, APIs, Web).
- Remove unnecessary columns and keep only needed ones.
- Change data types (text, number, date, boolean, etc.).
- Remove duplicates, blank rows, and errors.
- Filter rows based on conditions.
- Merge (JOIN) and Append (UNION) tables.
- Group rows and calculate aggregations.
- Pivot and Unpivot columns.
- Split columns by delimiter or position.
- Replace values and fill down/up.
- Create custom columns using M expressions.

Q20. Is Power Query case-sensitive?

Answer:

Yes. String operations like Replace Values and comparison filters are case-sensitive by default. To handle case-insensitively, use Text.Lower() or Text.Upper() M functions to normalize the text casing before making comparisons.

SECTION 3: Data Cleaning Operations in Power Query

Q21. What does the TRIM transformation do in Power Query?

Answer:

The Trim transformation removes all leading and trailing spaces from text values in a column. It does NOT remove spaces within the middle of a text string. For example, ' John ' becomes 'John'. In DAX, the TRIM() function also removes extra spaces between words, keeping only single spaces between words.

> **Tip:** Always Trim text columns after importing from Excel or CSV files, as they often contain hidden spaces that cause incorrect grouping and filtering.

Q22. What does the CLEAN transformation do in Power Query?

Answer:

The Clean transformation removes non-printable characters from text — characters that are invisible but cause issues in data processing (like newline characters, tab characters, control characters). These often sneak in when data is copied from web pages, PDFs, or legacy systems. Use Clean together with Trim for thorough text cleaning.

Q23. What are the different case transformations available in Power Query?

Answer:

Power Query provides three text case transformations under the Transform tab:

- UPPERCASE — Converts all characters to capitals. Example: 'john doe' → 'JOHN DOE'.
- lowercase — Converts all characters to small letters. Example: 'JOHN DOE' → 'john doe'.
- Capitalize Each Word (Proper Case) — Capitalizes the first letter of each word. Example: 'john doe' → 'John Doe'.
- In DAX: UPPER(), LOWER(), PROPER() perform the same transformations.

> **Tip:** Always standardize text casing before merging or grouping data to avoid mismatches like 'North' vs 'north' being treated as two different regions.

Q24. What is the difference between Remove Duplicates and Remove Errors in Power Query?

Answer:

These are two different cleaning operations:

- Remove Duplicates — Removes rows where the selected column(s) have identical values. Keeps only the first occurrence of each unique combination.
- Remove Errors — Removes entire rows that contain error values (like #Error, #NULL, division-by-zero errors) in the selected column.
- Remove Duplicates is used for deduplication. Remove Errors is used when bad data causes transformation errors.

Q25. What is the difference between Remove Columns and Remove Other Columns in Power Query?

Answer:

Both remove columns but work in opposite ways:

- Remove Columns — Deletes specifically the columns you select.
- Remove Other Columns — Keeps only the columns you select and deletes everything else.
- Best Practice: Use 'Remove Other Columns' when you want to keep only a small set of columns from a large table — it is more future-proof because new columns added to the source won't accidentally get included.

Q26. What is the difference between DISTINCT and UNIQUE in Power Query?

Answer:

These two operations appear similar but work differently:

- Remove Duplicates (Distinct) — Keeps only one instance of each duplicate row. If a value appears 3 times, it keeps the first occurrence and removes the other 2.
- Keep Duplicates (Unique inverse) — Keeps ONLY the rows that appear more than once (opposite of distinct).
- In DAX: DISTINCT() returns unique values ignoring the current filter context. VALUES() returns unique values respecting the filter context and may include a blank row for integrity issues.
- Use DISTINCT() when you need a clean unique list. Use VALUES() inside measures where filter context matters.

Q27. What are the different column types (data types) in Power Query?

Answer:

Power Query supports the following data types for columns:

- Text — Alphanumeric strings. Example: customer names, product codes.
- Whole Number — Integers with no decimal. Example: quantity, count.
- Decimal Number — Numbers with decimal places. Example: price, revenue.
- Fixed Decimal Number (Currency) — Decimal up to 4 decimal places. Best for financial values.
- Date — Date only without time component.
- Time — Time only without date component.
- Date/Time — Combined date and time.
- Date/Time/Timezone — Date/time with timezone offset.
- Duration — Length of time between two dates/times.
- Boolean — True or False values only.
- Binary — File content or binary data.
- Any — No enforced type (avoid using in production models).

> **Tip:** Always explicitly set correct data types in Power Query. Wrong types (like numbers stored as text) will cause DAX aggregations like SUM to fail.

Q28. What is the Replace Values operation and when do you use it?

Answer:

Replace Values allows you to find a specific value in a column and replace it with another value. Use cases include: fixing typos (e.g., 'Mombai' → 'Mumbai'), replacing null values with defaults (e.g., null → 0 or null → 'Unknown'), standardizing codes or abbreviations (e.g., 'M' → 'Male'), and cleaning inconsistent category labels.

Q29. What is Fill Down and Fill Up in Power Query?

Answer:

Fill Down and Fill Up propagate non-null values to fill blank/null cells in a column:

- Fill Down — Takes the last non-null value above and fills it into the null cells below it. Useful when a category label only appears once at the top of a group.
- Fill Up — Takes the first non-null value below and fills it into the null cells above it.
- Example: A merged Excel cell for 'Region: North' spans 5 rows but only has the value in the first row. Fill Down copies 'North' to all 5 rows.

Q30. What is Split Column operation in Power Query?

Answer:

Split Column breaks a single text column into multiple columns based on a separator or fixed position:

- By Delimiter — Splits on a character like comma, space, dash, or pipe. Example: 'John,Doe' → 'John' | 'Doe'.
- By Number of Characters — Splits after a fixed number of characters. Example: First 3 characters as one column, rest as another.
- By Position — Splits at specific character positions.
- By Lowercase/Uppercase Transition — Splits where case changes.
- Common use: Splitting full name into first and last name, splitting date strings.

Q31. What is the Group By operation in Power Query?

Answer:

Group By aggregates data by grouping rows that share the same value in one or more columns, and calculates summary statistics for each group. It is equivalent to SQL's GROUP BY statement. You specify the grouping columns and the aggregation operation (Sum, Average, Count, Min, Max, All Rows). Example: Group by Region and Sum the Sales column to get total sales per region.

Q32. What is the difference between Keep Rows and Remove Rows in Power Query?

Answer:

Both filter your data but from opposite directions:

- Remove Rows — Options include: Remove Top Rows, Remove Bottom Rows, Remove Alternate Rows, Remove Duplicates, Remove Blank Rows, Remove Errors.
- Keep Rows — Options include: Keep Top Rows, Keep Bottom Rows, Keep Range of Rows.
- Use Remove Rows to exclude headers accidentally loaded, remove total rows at the bottom of Excel sheets, or remove error-causing rows.
- Use Keep Rows when you only need a sample or specific range of data.

SECTION 4: Data Types — Categorical & Numerical

Q33. What are the two main types of data in data analysis?

Answer:

Data is broadly classified into two types:

- Categorical Data — Data that represents categories or groups. Cannot be measured numerically in a meaningful way. Examples: Gender, Country, Product Category, Color.
- Numerical Data — Data that represents quantities and can be measured. Can be added, averaged, or used in calculations. Examples: Sales Amount, Age, Temperature, Quantity.

Q34. What are Nominal and Ordinal data types in Power BI?

Answer:

These are the two subtypes of Categorical data:

- Nominal Data — Categories with NO natural order or ranking. One category is not greater or less than another. Examples: Country names (India, USA, UK), Gender (Male, Female), Product Category (Electronics, Clothing).
- Ordinal Data — Categories WITH a natural order or ranking. The order matters but the difference between values is not defined. Examples: Customer Rating (1 star, 2 star, 3 star), Education Level (High School, Undergraduate, Postgraduate), Survey Responses (Strongly Disagree, Disagree, Neutral, Agree, Strongly Agree).
- In Power BI: Use ordinal data for sorted slicers, ranked visuals, or ordered matrix rows.

> **Tip:** Nominal data should NEVER be used in calculations like average. Ordinal data can be sorted but averaging (e.g., average star rating) needs to be done carefully.

Q35. What are Discrete and Continuous numerical data types?

Answer:

These are the two subtypes of Numerical data:

- Discrete Data — Countable, whole number values. Cannot have fractional values. Examples: Number of customers (you can't have 2.5 customers), Number of orders, Count of products.
- Continuous Data — Can take ANY value within a range, including decimals and fractions. Examples: Sales revenue (₹1,234.56), Temperature (36.6°C), Distance (5.73 km), Weight.
- In Power BI: Discrete data maps to Whole Number type. Continuous data maps to Decimal Number type.
- Histograms and bin grouping are typically used with continuous data. Count charts are used with discrete data.

> **Tip:** Understanding data types helps you choose the right visualizations. Bar charts for discrete/categorical, line charts for continuous over time, scatter charts for continuous vs continuous.

Q36. How does Power BI handle data type classification for visuals?

Answer:

Power BI automatically classifies fields into different buckets based on their data type:

- Text fields — Treated as categorical dimensions. Placed in Axis or Legend fields.
- Numeric fields — Treated as measures by default (aggregated with SUM). Placed in Values fields.
- Date/Time fields — Treated as a hierarchy (Year > Quarter > Month > Day) automatically.
- Boolean fields — True/False treated as categorical.
- You can change how a field is treated by setting its 'Summarization' property or creating explicit DAX measures.

Q37. What is data granularity and why does it matter in Power BI?

Answer:

Granularity refers to the level of detail or specificity of data in a table. High granularity means each row represents a single transaction or event (e.g., one row per sale). Low granularity means data is already summarized (e.g., one row per month). In Power BI, it is best to import the highest granularity data (most detailed level) and let DAX aggregate it — this gives you maximum flexibility in your reports. Importing pre-aggregated data limits what you can analyze.

SECTION 5: Data Modeling & Relationships

Q38. What is a Data Model in Power BI?

Answer:

A Data Model is the structured blueprint of tables and their relationships in Power BI. It uses the VertiPaq in-memory columnar database engine. A good data model enables accurate cross-table calculations, fast visual rendering, and clean DAX formulas. Think of the data model as the 'backbone' of your Power BI report.

Q39. What is the difference between a Fact Table and a Dimension Table?

Answer:

These are two types of tables in a data model:

- Fact Table — Contains transactional/measurable data (sales amounts, order quantities, revenue). Has many rows. Example: FactSales, FactOrders.
- Dimension Table — Contains descriptive attributes used for filtering or grouping (product names, customer details, dates). Has fewer, unique rows. Example: DimProduct, DimCustomer, DimDate.
- The Fact table stores 'what happened'. Dimension tables store 'who, what, where, when' context.
- Fact tables contain foreign keys that link to dimension table primary keys.

Q40. What is Star Schema vs Snowflake Schema? Which is preferred for Power BI?

Answer:

These are two data modeling patterns:

- Star Schema — Fact table in the center, directly connected to all dimension tables. Denormalized, simple joins, fast performance. Looks like a star shape.
- Snowflake Schema — Dimension tables are further normalized (split into sub-dimension tables). More complex joins, slower performance, but less data redundancy. Looks like a snowflake.
- Star Schema is the recommended and preferred approach for Power BI because it offers better query performance, simpler DAX formulas, and easier maintenance.
- Example: In a grocery store scenario, Sales fact table connects directly to Customer, Product, Date, and Region dimension tables — that is Star Schema.

> **Tip:** Always recommend Star Schema for Power BI in interviews.

Q41. What are relationship cardinality types in Power BI?

Answer:

Cardinality describes how rows in one table relate to rows in another:

- One-to-Many (1:*) — Most common. One row in DimProduct matches many rows in FactSales. Filter flows from the 'one' side to the 'many' side.
- Many-to-One (*:1) — Same as 1:M but described from the other direction.
- One-to-One (1:1) — Each row matches exactly one row in the other table. Rare — often means the tables should be merged.
- Many-to-Many (*:*) — Both sides have duplicate keys. Use a bridge table or composite model. Handle with caution as it can cause double-counting.

Q42. What is the difference between Import Mode and DirectQuery Mode?

Answer:

These are two ways to bring data into Power BI:

- Import Mode — Data is fully loaded into Power BI's in-memory VertiPaq engine. Faster performance. Supports all DAX functions. Requires scheduled refresh. Best for small-to-medium datasets.
- DirectQuery Mode — Data stays in the source database. Queries run live on every interaction. Always shows real-time data. Slower performance. Limited DAX support. Best for large datasets or real-time analytics.
- Use Import Mode for fast interactive dashboards. Use DirectQuery for live inventory tracking, real-time financial dashboards, or data governance requirements.

Q43. What is Cross-Filter Direction and when should you use Bidirectional?

Answer:

Cross-Filter Direction controls how filters propagate across a relationship:

- Single Direction (default) — Filters flow only from the 'one' side (dimension) to the 'many' side (fact). Recommended for most models.
- Both / Bidirectional — Filters can flow in both directions. Useful in M:M models with bridge tables, or when slicers on both sides need to work together.
- Avoid bidirectional when: it creates circular dependencies, working with large datasets where it reduces performance, or a simple single-direction filter is sufficient.
- Alternative: Use CROSSFILTER() DAX function to temporarily enable bidirectional filtering inside a specific measure.

Q44. What is the difference between Active and Inactive relationships?

Answer:

Power BI allows only one active relationship between two tables at a time. The active relationship is automatically used in all calculations. Additional relationships between the same tables can exist but are marked inactive. Inactive relationships can be temporarily activated inside a DAX measure using the USERELATIONSHIP() function. Example: A Sales table with both OrderDate and ShipDate — only one can be the active relationship to the Date table; the other is inactive and used via USERELATIONSHIP().

Q45. What are Role-Playing Dimensions in Power BI?

Answer:

A Role-Playing Dimension occurs when a single dimension table (like DimDate) needs to connect to multiple date fields in a fact table (OrderDate, ShipDate, DeliveryDate). The solution is to create duplicate copies of the date table (Date_Ordered, Date_Shipped) and connect each to the relevant column. Then use USERELATIONSHIP() in DAX measures to activate the inactive relationships when needed.

SECTION 6: DAX — Data Analysis Expressions

Q46. What is DAX and what are its three fundamental concepts?

Answer:

DAX (Data Analysis Expressions) is a collection of functions, operators, and constants used in formulas to calculate and return values. It helps create new information from data you already have. The three fundamental concepts are:

- Syntax — How DAX formulas are written. Example: Total Sales = SUM(Sales[Revenue]).
- Functions — Pre-built formulas (SUM, CALCULATE, IF, DATEADD) that perform specific calculations.
- Context — Determines how a formula is evaluated. Row Context (one row at a time) and Filter Context (based on applied filters).

Q47. What is the difference between a Calculated Column and a Measure?

Answer:

Both use DAX but serve different purposes:

- Measure — Dynamic, aggregated calculation. Computed on-the-fly based on report filters. NOT stored in the model. Efficient. Responds to slicers and filters. Example: Total Sales = SUM(Sales[Revenue]).
- Calculated Column — Static, row-by-row calculation. Computed at data refresh and STORED in the model. Increases model size. Does not respond dynamically to filters. Example: Profit = Sales[Revenue] - Sales[Cost].
- Use Measures for anything shown in visuals (charts, cards, KPIs).
- Use Calculated Columns for row-level attributes needed for filtering or slicing (e.g., profit tier, full name).

Q48. What is Row Context vs Filter Context in DAX?

Answer:

Context is one of the most important concepts in DAX:

- Row Context — Evaluates a formula for each individual row. Present in Calculated Columns and iterator functions (SUMX, FILTER). Think: 'What is the value for THIS row?'
- Filter Context — The set of data visible when a measure is calculated, based on slicers, visual filters, and CALCULATE() filters. Think: 'What data is currently filtered in?'
- Example of Row Context: Profit = Sales[Revenue] - Sales[Cost] — each row uses its own Revenue and Cost values.
- Example of Filter Context: When a slicer selects 'North Region', a SUM measure only adds up North Region rows.

Q49. What is CALCULATE() and why is it the most important DAX function?

Answer:

CALCULATE() evaluates an expression in a MODIFIED filter context. It is the most powerful and most-used DAX function because it can override, add, or remove filters to compute values that would otherwise be impossible. Syntax: CALCULATE(expression, filter1, filter2, ...). Examples:

- North Sales = CALCULATE(SUM(Sales[Revenue]), DimRegion[Region] = 'North') — overrides context to show only North sales.
- YTD Sales = CALCULATE(SUM(Sales[Revenue]), DATESYTD(Date[Date])) — restricts to year-to-date dates.
- % of Total = DIVIDE(SUM(Sales[Revenue]), CALCULATE(SUM(Sales[Revenue]), ALL(Sales)), 0) — removes all filters for total.

Q50. What is the difference between SUM() and SUMX()?

Answer:

These two aggregation functions work differently:

- SUM(column) — Simply adds all values in a single numeric column. Fast and efficient. Does not perform row-by-row calculations. Example: Total Revenue = SUM(Sales[Revenue]).
- SUMX(table, expression) — Iterates row-by-row over a table, evaluates an expression for each row, then sums all the results. Use when you need to compute something per row first. Example: Total Value = SUMX(Sales, Sales[Quantity] * Sales[UnitPrice]).
- SUM() cannot multiply two columns row-by-row before summing. SUMX() can handle this.

> **Tip:** Use SUM() for simple column totals. Use SUMX() when the value must be calculated per row before aggregating.

Q51. What is the difference between ALL() and ALLEXCEPT()?

Answer:

Both are filter-removal functions used with CALCULATE():

- ALL(table/column) — Removes ALL filters from the specified table or column. Frequently used in % of total calculations to get the grand total denominator.
- ALLEXCEPT(table, column1, column2...) — Removes all filters EXCEPT the specified columns. Useful when you want to keep certain filters (like Region) while ignoring others.
- Example: % of Total = DIVIDE(SUM(Sales[Revenue]), CALCULATE(SUM(Sales[Revenue]), ALL(DimProduct)), 0) — this calculates each product's share of ALL products' total.

Q52. What is the difference between DISTINCT() and VALUES() in DAX?

Answer:

Both return unique values but behave differently:

- DISTINCT(column) — Returns unique values from a column. Strictly ignores blank rows and the current filter context. Always returns all distinct values regardless of what filters are applied.
- VALUES(column) — Returns unique values visible in the CURRENT filter context. Also includes a blank row if there are referential integrity issues (unmatched keys). Respects filters.
- Use VALUES() inside most DAX calculations where filter context matters. Use DISTINCT() when you need to explicitly exclude blank rows or ignore filter context.

Q53. What is the difference between CALCULATE() and CALCULATETABLE()?

Answer:

These two functions modify filter context but return different types of results:

- CALCULATE() — Evaluates an expression and returns a SCALAR (single) value with modified filters. Example: Total Electronics Sales = CALCULATE(SUM(Sales[Amount]), Products[Category]='Electronics') — returns a single number.
- CALCULATETABLE() — Returns an entire filtered TABLE instead of a scalar value. Used as input for other table functions. Example: Electronics_Sales = CALCULATETABLE(Sales, Products[Category]='Electronics') — returns a filtered table of rows.

Q54. What is the SUMMARIZE() function in DAX?

Answer:

SUMMARIZE() creates a new summary table grouped by one or more columns with optional aggregations. It is similar to SQL's GROUP BY statement and is used as a virtual table inside other DAX expressions. Syntax: SUMMARIZE(table, groupByColumn1, groupByColumn2, 'NewColumnName', expression). Example: Sales_Summary = SUMMARIZE(Sales, Product[Category], 'Total Sales', SUM(Sales[Revenue])) — groups sales by category and calculates total per category.

Q55. What is DIVIDE() and why should you use it instead of the / operator?

Answer:

DIVIDE(numerator, denominator, alternate_result) is a safe division function that returns the alternate_result (default: BLANK) when the denominator is zero or blank, instead of throwing a division-by-zero error. The / operator will cause an error in such cases. Example: Safe Margin = DIVIDE([Total Profit], [Total Revenue], 0). Always use DIVIDE() in production DAX formulas for robust, error-free calculations.

Q56. What are VAR and RETURN in DAX and why should you use them?

Answer:

VAR declares a variable in DAX to store an intermediate result, and RETURN specifies the final output. Benefits:

- Readability — Breaks complex formulas into clearly named parts.
- Performance — The sub-expression is only evaluated once, even if used multiple times.
- Debugging — You can temporarily change RETURN to return a VAR value to inspect it.
- Example: Profit Margin = VAR TotalRev = SUM(Sales[Revenue]) VAR TotalCost = SUM(Sales[Cost]) RETURN DIVIDE(TotalRev - TotalCost, TotalRev, 0)

Q57. What are the differences between MAX(), MAXA(), and MAXX() in DAX?

Answer:

These three functions find maximum values but work differently:

- MAX(column) — Returns the largest value in a numeric, date, or text column. Ignores logical values (TRUE/FALSE) and blanks.
- MAXA(column) — Same as MAX() but also handles logical values. TRUE is treated as 1, FALSE as 0, blanks as 0.
- MAXX(table, expression) — Iterates row-by-row over a table, evaluates an expression for each row, and returns the largest result. Example: Max Adjusted Revenue = MAXX(Sales, Sales[Revenue] * (1 - Sales[Discount]/100)).

Q58. What is SWITCH() and how is it different from IF()?

Answer:

Both are logical functions but work best in different situations:

- IF(condition, result_if_true, result_if_false) — Best for simple true/false conditions. Can be nested for multiple conditions but becomes hard to read.
- SWITCH(expression, value1, result1, value2, result2, ..., else_result) — Evaluates an expression against multiple values and returns the matching result. Cleaner alternative to nested IF statements.
- SWITCH(TRUE(), ...) — Advanced pattern that allows range-based conditions. Example: Revenue Band = SWITCH(TRUE(), Sales[Revenue]>=3000, 'Premium', Sales[Revenue]>=2000, 'High', Sales[Revenue]>=1000, 'Medium', 'Low').
- Use IF for binary (yes/no) decisions. Use SWITCH for multiple category assignments.

Q59. What is the RELATED() function used for?

Answer:

RELATED() fetches a value from a related table (on the 'one' side of a relationship) for use in a Calculated Column on the 'many' side. It requires an active relationship between the tables. Example: In a Sales table, if you want to add the Product Category from DimProduct, you write: Product Category = RELATED(DimProduct[Category]). This brings the category value from DimProduct into each row of the Sales calculated column.

SECTION 7: Time Intelligence Functions

Q60. What are Time Intelligence functions and what do they require?

Answer:

Time Intelligence functions allow you to perform date-based calculations like Year-to-Date, Month-to-Date, same period last year, and running totals. Three requirements:

- A dedicated Date/Calendar table in the model.
- The Date table must be marked as a 'Date Table' in Power BI (Modeling tab > Mark as Date Table).
- The Date column must be continuous with no missing dates (every day from start to end year).
- Common Time Intelligence functions: TOTALYTD(), TOTALMTD(), TOTALQTD(), SAMEPERIODLASTYEAR(), DATEADD(), DATESYTD(), DATESMTD(), DATESQTD().

Q61. What is SAMEPERIODLASTYEAR() and why is it important?

Answer:

SAMEPERIODLASTYEAR() returns the same date period from the previous year, enabling year-over-year comparisons. You cannot meaningfully compare January 2025 with December 2024 because December is affected by Christmas and year-end factors. The correct comparison is January 2025 vs January 2024. Example: Sales LY = CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Date[Date])). YoY Growth % = DIVIDE([Total Sales] - [Sales LY], [Sales LY], 0).

Q62. What is the difference between DATESYTD() and TOTALYTD()?

Answer:

Both calculate Year-to-Date values but work differently:

- DATESYTD(Date[Date]) — Returns a TABLE of dates from January 1 to the current date. Used inside CALCULATE().
- TOTALYTD(expression, Date[Date]) — A shortcut that combines CALCULATE + DATESYTD in one step. Returns a scalar value directly. Easier to write.
- YTD with DATESYTD: YTD Sales = CALCULATE(SUM(Sales[Revenue]), DATESYTD(Date[Date]))
- YTD with TOTALYTD: YTD Sales = TOTALYTD(SUM(Sales[Revenue]), Date[Date]) — same result.

Q63. What is the difference between SAMEPERIODLASTYEAR() and DATEADD()?

Answer:

Both shift date periods but with different flexibility:

- SAMEPERIODLASTYEAR(Date[Date]) — Returns the exact same period from the previous year. Only works for year-back comparisons.
- DATEADD(Date[Date], -1, YEAR) — More flexible. Can shift by any number of days, months, quarters, or years in either direction. Positive values go forward; negative values go backward.
- Use SAMEPERIODLASTYEAR() for standard Year-over-Year comparison. Use DATEADD() for custom time shifts like 'sales 3 months ago' or '2 quarters forward'.

Q64. What is the need for a Date Master Table in Power BI?

Answer:

Time intelligence functions require a reference point — today's date — to calculate periods like 'last month', 'last quarter', or 'year-to-date'. The Date Master Table is a dedicated calendar table that provides this reference. It tells Power BI what the correct and complete date range is. Without a proper Date table, time intelligence functions may produce incorrect results or fail entirely. A complete Date table has one row for every day in the range, with columns like Year, Month, Quarter, Week, Day Type, etc.

Q65. What is DATEDIFF() and what is DATESINPERIOD()?

Answer:

These two functions work with date ranges:

- DATEDIFF(start_date, end_date, interval) — Calculates the difference between two dates in the specified unit (DAY, WEEK, MONTH, QUARTER, YEAR). Example: Delivery Days = DATEDIFF(Orders[OrderDate], Orders[DeliveryDate], DAY).
- DATESINPERIOD(dates, start_date, number_of_intervals, interval) — Returns a table of dates for a specific time window. Use negative number_of_intervals for past periods. Example: Last 3 Months = CALCULATE(SUM(Sales[Revenue]), DATESINPERIOD(Sales[Date], MAX(Sales[Date]), -3, MONTH)).

SECTION 8: Visualizations, Filters & Slicers

Q66. What are the different types of visualizations in Power BI?

Answer:

Power BI provides a wide range of built-in visualizations:

- Bar & Column Charts — Compare values across categories (Clustered, Stacked, 100% Stacked).
- Line & Area Charts — Show trends over time.
- Pie & Donut Charts — Show parts of a whole. Limit to max 5-6 slices for readability.
- Card & Multi-row Card — Display single or multiple KPI numbers.
- Table & Matrix — Display detailed data or cross-tab pivot-style summaries.
- Slicer — Interactive on-canvas filter for end users.
- Map & Filled Map — Geographic data visualization.
- Scatter & Bubble Charts — Show correlation between two measures.
- Gauge & KPI Visuals — Show actual vs target performance.
- Waterfall Chart — Show how individual positive/negative values contribute to a total.
- Funnel Chart — Show stages in a process with drop-off.

Q67. What are the three filter levels in Power BI and how do they differ?

Answer:

Filters can be applied at three levels in the Filters pane:

- Visual Level Filter — Applies only to a single specific visual on the page. Does not affect any other visual.
- Page Level Filter — Applies to ALL visuals on the current report page only. Does not affect other pages.
- Report Level Filter — Applies to ALL visuals on ALL pages of the entire report. Useful for maintaining consistent filters like 'only active customers' across the whole report.

Q68. What are Slicers in Power BI and what types are available?

Answer:

Slicers are interactive visual filters placed on the canvas that allow end-users to dynamically filter report data without opening the Filters pane. Types of slicers:

- List Slicer — Shows items as a list with checkboxes.
- Dropdown Slicer — Saves space with a collapsible dropdown.
- Date Range Slicer — Filters data using a date range (slider, between, before/after).
- Numeric Range Slicer — Filters within a numeric range.
- Hierarchy Slicer — Multi-level filter (e.g., Country > State > City).
- Relative Date Slicer — Dynamic filter like 'last 30 days' or 'this quarter'.
- Tile/Button Slicer — Buttons for single or multi-select filtering.

Q69. What are Bookmarks in Power BI?

Answer:

A Bookmark captures the complete state of a report page — including all filter selections, slicer values, and visual visibility status. Bookmarks are used with Buttons to create: interactive navigation between views, toggle show/hide visuals, storytelling with guided navigation, and what-if scenario switching. They make reports feel like interactive applications instead of static charts.

Q70. What are Custom Visuals in Power BI?

Answer:

Custom Visuals extend Power BI's built-in visual gallery. They are downloaded from Microsoft AppSource (the official marketplace). Three types:

- Certified Custom Visuals — Reviewed and approved by Microsoft for security and performance. Safe to use in production.
- Non-Certified Custom Visuals — Developed by third-party vendors. Not fully vetted by Microsoft. Cannot be exported in some scenarios.
- Custom Developed Visuals — Built by developers using Power BI Visuals SDK. For unique internal business needs.
- Always prefer Microsoft-certified custom visuals in production reports for security and compatibility.

Q71. What is Drillthrough in Power BI?

Answer:

Drillthrough allows users to right-click on a data point in a visual and navigate to a separate detail report page that is pre-filtered for that selected item. To set up drillthrough: create a new report page, drag a field to the Drillthrough Filters well on that page, and add detailed visuals. Power BI automatically adds a Back button. Use case: Click on 'North Region' in a summary chart and drillthrough to a detailed North Region transactions page.

Q72. What is Visual Interaction and how do you change it?

Answer:

By default, clicking a data point in one visual filters all other visuals on the page. You can control this behavior per visual pair via Format tab > Edit Interactions. For each pair, you can set:

- Filter — The selected visual restricts the other visual to matching data.
- Highlight — The selected visual dims non-matching data in the other visual (does not filter out).
- None — The selected visual has no effect on the other visual.
- Use case: Prevent a map from filtering a summary card when clicking a region.

Q73. How do you create dynamic titles in Power BI visuals?

Answer:

Create a DAX measure that returns the desired title text dynamically, then bind it to the visual's Title property via the Format pane's conditional formatting option (fx button next to Title). Example: Report Title = 'Sales Report - ' & SELECTEDVALUE(Date[Year], 'All Years'). When the user selects 2024 from a slicer, the title automatically updates to 'Sales Report - 2024'.

SECTION 9: Power BI Service, Security & Gateway

Q74. What is the difference between My Workspace and a Workspace in Power BI Service?

Answer:

Think of it like a personal notebook vs a shared team notebook:

- My Workspace — Personal sandbox space. Only you can view and edit it. Good for testing, personal development, and draft work. Not for team sharing.
- Workspace — Shared team space. Multiple users can collaborate, publish, and view reports. Best for production projects that need to be shared with the team or organization.
- You can have one or more workspaces for different projects or teams.

Q75. What are the Workspace Roles in Power BI Service?

Answer:

Power BI Service has four workspace roles with different permission levels:

- Admin — Full control. Can add/remove users, manage permissions, publish, and delete all content. Highest access.
- Member — Can create, edit, and publish content. Can add Contributors. Cannot manage permissions.
- Contributor — Can create and edit reports and dashboards. Cannot publish or manage users.
- Viewer — Read-only access. Can view reports and dashboards only. Cannot edit or publish. Lowest access.
- Assign: Admin for IT Managers, Member for Report Developers, Contributor for Analysts, Viewer for Business End Users.

Q76. What is Row-Level Security (RLS) in Power BI?

Answer:

RLS (Row-Level Security) restricts data access so that users see only the rows they are authorized to view. Example: India users see only India data; US users see only US data. Two types:

- Static RLS — Manually define DAX filter rules for each role in Power BI Desktop. Example: [Region] = 'North'. Assign users to roles in Power BI Service. Disadvantage: must manually maintain user-role assignments.
- Dynamic RLS — Uses USERPRINCIPALNAME() DAX function to match the logged-in user's email against a security mapping table. Automatically filters data based on user identity. More scalable than static RLS.
- Both types must be created in Power BI Desktop first, then published and assigned in Power BI Service.

Q77. What is a Gateway in Power BI and what are its types?

Answer:

A Gateway is a bridge that enables secure, encrypted data transfer between on-premises data sources (SQL Server, Excel, SharePoint, ERP systems) and Power BI Service (cloud). Without a gateway, the cloud-based Power BI Service cannot refresh data stored on local/on-premise servers. Two types:

- Personal Gateway — Designed for single-user access only. Supports scheduled refresh. No DirectQuery support. Works with Power BI only. Best for personal practice and individual use.
- Standard (Enterprise) Gateway — Supports multiple users. Allows both DirectQuery and Scheduled Refresh. Can be used with Power BI, PowerApps, Power Automate, and Azure Logic Apps. Best for organization-wide production use.

Q78. What are the types of data refresh in Power BI Service?

Answer:

Power BI supports several refresh methods:

- Scheduled Refresh — Define frequency and time slots. Up to 8 refreshes/day for Pro, 48/day for Premium. Runs automatically without user action.
- On-Demand Refresh — Manual refresh triggered by clicking Refresh Now in Power BI Service.
- Incremental Refresh — Refreshes only new or changed data rather than the full dataset. Ideal for large tables with historical data. Requires Premium capacity for full support.
- Real-Time Refresh — DirectQuery mode always shows live data without any refresh.

Q79. What is Dataflow in Power BI Service?

Answer:

Dataflow is the equivalent of Power Query Editor but inside Power BI Service (cloud). It allows you to perform ETL transformations, create reusable datasets, and share cleaned data tables across multiple workspaces and reports without needing Power BI Desktop. In other words, Dataflow = Power Query Editor in the cloud. It is especially useful for centralized data preparation in teams so that multiple reports share the same clean data source.

SECTION 10: Performance Optimization

Q80. What are common causes of slow Power BI reports and how do you fix them?

Answer:

Common performance problems and their solutions:

- Problem: Too many unnecessary columns/rows imported. Fix: Remove unused columns in Power Query before loading.
- Problem: Complex nested DAX (nested IFs, unoptimized FILTER). Fix: Use SWITCH(TRUE()) instead of nested IF; use boolean filters instead of FILTER() for simple conditions.
- Problem: Too many visuals on one page. Fix: Limit visuals per page; use Bookmarks and Drillthrough for detail.
- Problem: Calculated Columns instead of Measures. Fix: Prefer Measures — they are computed on demand, not stored in memory.
- Problem: Auto Date/Time enabled (Power BI creates hidden date tables for every date column). Fix: Disable via File > Options > Data Load > uncheck Auto Date/Time.
- Problem: High-cardinality text columns in visuals. Fix: Use lower-cardinality groupings; avoid unique ID columns in visuals.
- Problem: Bidirectional relationships everywhere. Fix: Use single-direction where possible; bidirectional only when necessary.

Q81. What is the Performance Analyzer in Power BI?

Answer:

Performance Analyzer (View tab > Performance Analyzer) records how long each visual takes to render when a report page loads or a filter is applied. It shows DAX query time, visual rendering time, and other events — helping you identify which visuals or measures are slow so you can optimize them. You can export the results to analyze bottlenecks in detail.

Q82. What is VertiPaq in Power BI and why does it matter?

Answer:

VertiPaq is Power BI's in-memory columnar storage engine. It compresses data column-by-column and stores it in RAM for extremely fast read access. Key facts:

- Lower column cardinality (fewer unique values) = better compression = smaller model = faster performance.
- This is why you should use integer IDs instead of long text strings for key columns.
- Removing high-cardinality columns like full timestamps when only the date is needed significantly reduces model size.
- The VertiPaq engine is what makes Import Mode much faster than DirectQuery.

SECTION 11: Scenario-Based Interview Questions

Q83. How would you build a Power BI report from scratch?

Answer:

Step-by-step approach to building a Power BI report:

- Step 1: Connect to data source (Excel/SQL/API) using Get Data in the Home tab.
- Step 2: Clean and transform data in Power Query (remove nulls, fix data types, rename columns, remove duplicates).
- Step 3: Build the data model — create relationships between tables, add a Date dimension table, design a Star Schema.
- Step 4: Write DAX measures (Total Sales, YTD, % of Total, % Growth, etc.).
- Step 5: Design report pages with charts, cards, slicers, and matrix visuals.
- Step 6: Format visuals with consistent colors, titles, data labels, and tooltips.
- Step 7: Publish to Power BI Service and set up scheduled refresh.
- Step 8: Configure RLS if needed and share with stakeholders or publish as an App.

Q84. A DAX measure is showing incorrect totals in a Matrix visual — how do you debug it?

Answer:

Systematic debugging approach for incorrect DAX totals:

- Check for incorrect CALCULATE usage — verify which filters are being applied or overridden.
- Use VAR to break the formula into intermediate steps and inspect each value.
- Add a Table visual with the grouping columns to see what values the formula returns row-by-row.
- Check for bidirectional cross-filter issues creating unexpected filter propagation.
- Verify the data model — wrong cardinality or missing relationships cause incorrect aggregations.
- Check if the measure uses FILTER() where a simpler boolean filter would work better.
- Look for context transition issues if the measure references other measures.

Q85. How would you handle data from multiple different sources in Power BI?

Answer:

Approach for combining multiple data sources:

- Use Power Query to connect to each source separately using Get Data.
- Use Merge (JOIN) in Power Query to combine tables with matching keys horizontally.
- Use Append (UNION) in Power Query to stack tables with the same structure vertically.
- Standardize key columns — ensure consistent naming, data types, and values (e.g., 'North' vs 'N') across sources before merging.
- Create a unified data model with clear relationships between all tables.
- Build a shared Date dimension table for consistent time intelligence across all sources.

Q86. How would you implement Year-over-Year growth analysis in a Power BI dashboard?

Answer:

Step-by-step implementation of YoY growth:

- Ensure you have a proper Date table marked as a Date Table in Power BI.
- Create measure: Total Sales = SUM(Sales[Revenue])
- Create measure: Sales LY = CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Date[Date]))
- Create measure: YoY Growth = [Total Sales] - [Sales LY]
- Create measure: YoY Growth % = DIVIDE([Total Sales] - [Sales LY], [Sales LY], 0)
- Add a Line Chart with Date on the X-axis and both Total Sales and Sales LY as lines for visual comparison.
- Add a KPI visual showing Total Sales as indicator, Sales LY as target, and Date as trend axis.
- Add conditional formatting to YoY Growth % — green for positive, red for negative.

Q87. A client wants a report where different managers see only their own team's data — how do you implement this?

Answer:

This is implemented using Dynamic Row-Level Security (RLS):

- Step 1: Create a User Access Table with columns: ManagerEmail, Team/Region.
- Step 2: Link the User Access Table to the main data table via the Team/Region column.
- Step 3: In Power BI Desktop, go to Modeling > Manage Roles and create a 'ManagerAccess' role.
- Step 4: In the User Access Table, add the DAX filter: [ManagerEmail] = USERPRINCIPALNAME()
- Step 5: Publish the report to Power BI Service.
- Step 6: In Power BI Service, assign each manager's email to the 'ManagerAccess' role.
- Step 7: When each manager logs in, USERPRINCIPALNAME() returns their login email and automatically filters to their data only.

Q88. How would you optimize a Power BI report that is loading very slowly?

Answer:

Systematic optimization approach:

- Run Performance Analyzer to identify which visuals and DAX queries are slowest.
- In Power Query: Remove unused columns and rows before loading. Push filters to the source query for query folding.
- In the Data Model: Disable Auto Date/Time. Ensure Star Schema design. Check for unnecessary bidirectional relationships.
- In DAX: Replace Calculated Columns with Measures where possible. Use DIVIDE() instead of /. Use VAR for repeated sub-expressions. Replace FILTER() with boolean filters for simple conditions.
- On Report Pages: Reduce number of visuals per page. Use Bookmarks to show/hide visuals instead of cramming all on one page. Use Drillthrough for detail.
- Consider switching from DirectQuery to Import Mode if real-time data is not mandatory.

Q89. What would you do if two tables have a Many-to-Many (M:M) relationship?

Answer:

Approach for handling Many-to-Many relationships:

- Option 1 (Recommended): Create a Bridge/Junction Table — an intermediate table that resolves M:M into two 1:M relationships. Example: Students and Courses have M:M; create an Enrollment table.
- Option 2: Use Composite Models — Power BI natively supports M:M relationships but it can cause double-counting issues. Use only if you understand the implications.
- Option 3: Use TREATAS() in DAX to virtually map a relationship for specific calculations.
- Always explain to the interviewer WHY M:M exists and how the bridge table solves it — this shows deep understanding.

Q90. How do you create a % of Total measure that works correctly with slicers?

Answer:

Creating a % of Total that respects slicer selections requires careful use of ALL() vs ALLSELECTED():

- % of Grand Total (ignores slicers): % Grand Total = DIVIDE(SUM(Sales[Revenue]), CALCULATE(SUM(Sales[Revenue]), ALL(Sales)), 0)
- % of Filtered Total (respects slicers — recommended): % of Selection = DIVIDE(SUM(Sales[Revenue]), CALCULATE(SUM(Sales[Revenue]), ALLSELECTED(Sales)), 0)
- ALLSELECTED() removes row/column context but keeps slicer context, so the denominator is the total of only the slicer-selected data.
- ALL() completely ignores all filters including slicers, giving % of the absolute grand total.

Q91. What KPIs are commonly tracked in Power BI dashboards and how are they implemented?

Answer:

Common KPIs across business domains:

- Sales: Total Revenue (SUM), Revenue vs Target (DIVIDE), Sales Growth % (YoY using SAMEPERIODLASTYEAR), Average Order Value (DIVIDE([Total Revenue], COUNTROWS(Sales))).
- Customer: Customer Count (DISTINCTCOUNT(Sales[CustomerID])), New vs Returning Customers, Churn Rate.
- Operations: On-Time Delivery % (DIVIDE(on-time count, total orders)), Defect Rate.
- Finance: Gross Margin % (DIVIDE([Gross Profit], [Revenue])), Budget vs Actual (DIVIDE([Actual], [Budget]) - 1).
- HR: Attrition Rate (DIVIDE([Leavers], [Headcount])), Average Tenure.

SECTION 12: Advanced Topics & Miscellaneous

Q92. What is the difference between bidirectional cross-filtering and CROSSFILTER() DAX function?

Answer:

Bidirectional cross-filtering is a relationship setting in the data model that permanently allows filters to flow in both directions between related tables. The CROSSFILTER() function is a DAX function used inside CALCULATE() to temporarily change the cross-filter direction for a specific calculation only. CROSSFILTER() is preferred over always-on bidirectional because it limits the impact to only where it is needed, avoiding performance and ambiguity issues elsewhere.

Q93. What is the difference between Scheduled Refresh and Incremental Refresh?

Answer:

Two important refresh strategies in Power BI:

- Scheduled Refresh — Refreshes the ENTIRE dataset at configured intervals (up to 8x/day for Pro, 48x/day for Premium). Simple to set up but time-consuming for large datasets.
- Incremental Refresh — Only refreshes NEW or CHANGED data since the last refresh. Dramatically faster for large tables. Requires defining RangeStart and RangeEnd parameters. Requires Premium capacity for full historical partitioning support.
- Use Incremental Refresh for tables with millions of rows where only the last few days/weeks change regularly.

Q94. What is Context Transition in DAX?

Answer:

Context Transition occurs when CALCULATE() is used inside a Calculated Column (which has row context). CALCULATE() automatically converts the current row context into an equivalent filter context, so aggregation functions like SUM() can work correctly. This is an advanced DAX concept: it means that inside a Calculated Column, wrapping an expression in CALCULATE() changes its evaluation from row-by-row to filtered aggregation. Understanding context transition is critical for writing correct measures that are referenced inside calculated columns.

Q95. What is SELECTEDVALUE() and HASONEVALUE()?

Answer:

These DAX functions detect what is selected in a slicer or filter:

- SELECTEDVALUE(column, default) — Returns the single selected value when exactly one value is filtered in a column. If no value or multiple values are selected, returns the default value.
- HASONEVALUE(column) — Returns TRUE if only one value is currently visible in the column (filtered to exactly one). Returns FALSE for none or multiple. Useful for conditional logic.
- Use case: Show a selected region name in a card: Selected Region = SELECTEDVALUE(DimRegion[Region], 'All Regions').

Q96. What is the difference between COUNTROWS(), COUNT(), COUNTA(), and DISTINCTCOUNT()?

Answer:

These are four different counting functions in DAX:

- COUNTROWS(table) — Counts the total number of rows in a table (including blank values). Most useful for row-level counts.
- COUNT(column) — Counts the number of non-blank NUMERIC values in a column.
- COUNTA(column) — Counts the number of non-blank values in a column of ANY data type (text, number, date).
- DISTINCTCOUNT(column) — Counts only the unique, non-blank values in a column. Used for counting unique customers, products, etc.
- Example: DISTINCTCOUNT(Sales[CustomerID]) gives the number of unique customers who made a purchase.

Q97. What are Text functions in DAX and when are they used?

Answer:

DAX provides many text manipulation functions:

- CONCATENATE() — Joins two text strings. CONCATENATEX() — Joins text from a column with a delimiter.
- FORMAT() — Converts numbers/dates to formatted text. Example: FORMAT(TODAY(), 'MMMM YYYY') returns 'April 2025'.
- LEFT(text, n), RIGHT(text, n) — Extracts n characters from left or right of text.
- MID(text, start, length) — Extracts a substring from a given position.
- LEN(text) — Returns the number of characters in text.
- TRIM(text) — Removes extra spaces (keeps single spaces between words).
- UPPER(), LOWER(), PROPER() — Change text casing.
- SUBSTITUTE(text, old, new) — Replaces all occurrences of a substring.
- REPLACE(text, start, n, new) — Replaces n characters at a specific position.
- SEARCH() — Case-insensitive substring position. FIND() — Case-sensitive substring position.

Q98. What is the difference between Power BI and Power Pivot for Excel?

Answer:

Key differences between the two tools:

- Power Pivot for Excel supports only single-direction relationships (one-to-many), and has only one Import Mode.
- Power BI Desktop supports bidirectional cross-filtering, Row-Level Security, calculated tables, DirectQuery mode, and many more import options.
- Power BI has a richer visualization engine with 30+ built-in visuals and AppSource marketplace.
- Power BI Service enables cloud sharing, collaboration, scheduled refresh, and mobile access — none of which exist in Excel Power Pivot.
- Use Power Pivot for Excel when working within an existing Excel workflow. Use Power BI for enterprise BI and sharing.

Q99. Where is data stored in Power BI?

Answer:

Data in Power BI is stored in two main locations in Azure:

- Azure Blob Storage — When users upload data (like Excel files), the raw data is stored here.
- Azure SQL Database — All metadata, report definitions, model artifacts, and system information are stored here.
- In Import Mode: Data is loaded into the VertiPaq in-memory engine inside the .pbix file and in Power BI Service's cloud cache.
- In DirectQuery Mode: Data remains in the original source database and is never copied into Power BI.

Q100. What are the best practices for Power BI report design?

Answer:

Key design best practices for professional Power BI reports:

- Use clear, readable fonts (e.g., Segoe UI) and consistent font sizes.
- Apply consistent brand colors and ensure sufficient contrast for accessibility.
- Use number formatting — thousand separators, currency symbols, and appropriate decimal precision.
- Use consistent date formats across all visuals.
- Ensure all charts have meaningful titles and axis labels.
- Place high-level KPIs at the top for quick insights, details lower down.
- Limit visuals per page — a clean, focused page is better than a crowded one.
- Use filters and slicers for interactivity rather than pre-filtering data.
- Avoid 3D charts — they distort data perception.
- Use pie charts sparingly — maximum 5-6 slices for readability.

QUICK REFERENCE: DAX Functions Cheat Sheet

Function	Category	Purpose
SUM(col)	Aggregation	Sum all values in a column
SUMX(table, expr)	Iterator	Sum a row-level expression across a table
AVERAGE(col)	Aggregation	Average of a numeric column
COUNT(col)	Aggregation	Count non-blank numeric values
COUNTROWS(table)	Aggregation	Count all rows in a table
DISTINCTCOUNT(col)	Aggregation	Count unique non-blank values
CALCULATE(expr, filters)	Filter	Evaluate expression with modified filter context
ALL(table/col)	Filter	Remove all filters from a table or column
ALLEXCEPT(table, col)	Filter	Remove all filters except specified columns
ALLSELECTED(table)	Filter	Remove row/column context but keep slicer context
FILTER(table, condition)	Filter	Returns a filtered table based on condition
RELATED(col)	Relationship	Fetch value from related table in a calculated column
USERELATIONSHIP(col1, col2)	Relationship	Activate an inactive relationship inside CALCULATE
DIVIDE(num, den, alt)	Math	Safe division — returns alt if denominator is 0
IF(cond, T, F)	Logical	Conditional logic
SWITCH(expr, v1, r1...)	Logical	Multi-value match — cleaner alternative to nested IFs
IFERROR(expr, alt)	Logical	Returns alt if expression produces an error
DATESYTD(dates)	Time Intel	Returns dates from Jan 1 to current date
TOTALYTD(expr, dates)	Time Intel	Year-to-Date shortcut
TOTALMTD(expr, dates)	Time Intel	Month-to-Date shortcut
SAMEPERIODLASTYEAR(dates)	Time Intel	Returns same period from previous year
DATEADD(dates, n, interval)	Time Intel	Shift dates by n periods (DAY/MONTH/QUARTER/YEAR)
DATEDIFF(d1, d2, interval)	Time Intel	Calculate difference between two dates
RANKX(table, expr, , order)	Iterator	Rank rows dynamically based on an expression
SUMMARIZE(table, col...)	Table	Create a grouped summary table
FORMAT(value, fmt)	Text	Convert date/number to formatted text string
CONCATENATE(t1, t2)	Text	Join two text strings into one
TRIM(text)	Text	Remove leading/trailing spaces
SELECTEDVALUE(col, default)	Filter	Return single selected value or default
VAR x = expr RETURN x	Variable	Store intermediate results for cleaner formulas